Technical Note:

1. Project Overview and Goals

The primary objective of this project is to enhance the security of the **Software Supply Chain**. In modern development, engineers rely heavily on **Third-Party Libraries (TPLs)** to speed up the process. However, these external components can introduce security risks if they contain unpatched vulnerabilities.

This project aims to develop a tool that can analyze a compiled Android application (the APK) and generate a **Software Bill of Materials (SBOM)**—an exhaustive list of all software components and their exact versions. Even when developers use techniques to hide these libraries, our goal is to identify them to detect any associated security vulnerabilities.

2. Security Fundamentals: CVE, SBOM, and Databases

To manage risk effectively, we must understand how vulnerabilities are tracked:

- **SBOM (Software Bill of Materials):** A formal, machine-readable inventory of all libraries and dependencies used to build a product.
- **CVE (Common Vulnerabilities and Exposures):** A unique, standardized identifier for a specific security flaw.
- **Vulnerability Databases:** We use the **NVD (National Vulnerability Database)** and **OSS Index** to check our SBOM against known threats.
- **CVSS (Common Vulnerability Scoring System):** A numerical score from 0.0 to 10.0 that indicates the severity of a vulnerability.

3. The Android Compilation Journey

An Android application goes through several states before it reaches the user. Understanding these transformations is essential for locating hidden data:

- **Source Code:** The original Java or Kotlin files.
- **Compilation:** Source code is first turned into Java bytecode (`.class` files).
- **DEXing:** The tool **D8** converts these classes into **DEX (Dalvik Executable) bytecode**. During this step, libraries are integrated directly into the bytecode of the application, losing their individual file boundaries.
- **The APK (Android Package):** The final container holding the DEX files, the **Manifest** (app metadata), and resources.
- **ART and JIT Execution:** Once installed, the **Android Runtime (ART)** may perform **Ahead-of-Time (AOT)** or **Just-in-Time (JIT)** compilation to translate bytecode into machine-specific code.

4. Technical Obstacles: Obfuscation and Transformation

The "locks" (verrous) in this project involve **Obfuscation : the intentional process of making code difficult to read to protect intellectual property.** and the loss of metadata during the build process:

- **Renaming:** Changing names like `SecurityLib` into meaningless characters like `a.b.c`.
- **Inlining:** The compiler replaces a function call with the actual code of that function, making the library "melt" into the main app code.
- **Fusion and Fission: Fusion** combines multiple functions into one, while **Fission** breaks one function into many smaller pieces.
- **Code Remaniement (Restructuring):** Altering the internal structure or the order of instructions to prevent matching with known library signatures.
- **Control Flow Flattening:** Scrambling the logical path of the program to confuse analysis tools.
- **Metadata & Provenance Loss:** During compilation, version numbers, signatures, and information about where the library came from (provenance) are often stripped away.

5. Identification and Analysis Techniques

Since we cannot rely on simple names, we use a research-based approach to identify libraries:

- **Static Analysis:** Examining the bytecode without running it using tools like **Jadx** or **Apktool.**
- **Binary Fingerprinting:** Comparing "hashes" of code blocks against a database of known libraries.
- **Class Structural Similarity:** Analyzing how classes relate to each other. This structure often remains consistent even when names are changed.
- **Code Similarity (Fuzzy Matching):** Using algorithms to compare **Control Flow Graphs (CFG)**. This looks at the "shape" of the code logic; even if names change, the shape often remains the same.
- **Feature Extraction:** Searching for unique "anchors" such as specific strings, error messages, or mathematical constants that obfuscators might miss**.**
- **Feature Language Models:** Using AI/ML models to recognize patterns and "dialects" in the bytecode that belong to specific libraries.
- **Multi-scale Fusion:** Combining different types of data (constants, control flow, and structural hierarchy) to create a stronger identification.

6. Implementation and Evaluation

The final prototype will focus on identifying libraries to detect CVEs.

- **Syft:** To scan the application and package the findings into an SBOM**.**
- **Trivy / Grype:** To scan the generated SBOM for vulnerabilities**.**
- **Version-Level Detection:** A CVE usually affects a specific **version interval**. Our tool must be precise enough to determine if the detected library falls within that dangerous range.
- **Transitive Dependency Detection:** Identifying libraries that are nested inside other libraries.
- **Validation:** We will evaluate the **Precision** (accuracy) and **Robustness** (resistance to obfuscation) of our tool using a **Ground Truth** dataset—applications where the original source code and library versions are already known.